

Università degli Studi di Milano
Corso di Laurea Magistrale in Informatica

Scheduling Policy-Driven di Task in Kubernetes con Vincoli di Protezione dei Dati

Candidato
Luca Milanesi
Matricola 55053A

Relatore
Prof.ssa Sara Foresti

Correlatore
Prof.ssa Pierangela Samarati

Anno Accademico 2025-2026

Scheduling Policy-Driven di Task in Kubernetes con Vincoli di Protezione dei Dati

Università degli Studi di Milano
Corso di Laurea Magistrale in Informatica

Scheduling Policy-Driven di Task in Kubernetes con Vincoli di Protezione dei Dati

Candidato
Luca Milanesi
Matricola 55053A

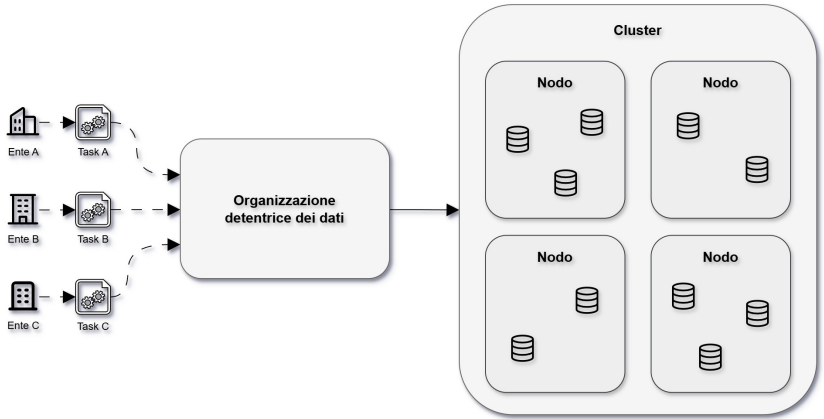
Relatore
Prof.ssa Sara Foresti

Correlatore
Prof.ssa Pierangela Samarati

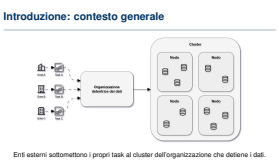
Anno Accademico 2025-2026

Buongiorno a tutti, sono Luca Milanesi e la mia tesi riguarda lo scheduling di task guidato da politiche con vincoli di protezione dei dati, implementato in Kubernetes.

Introduzione: contesto generale



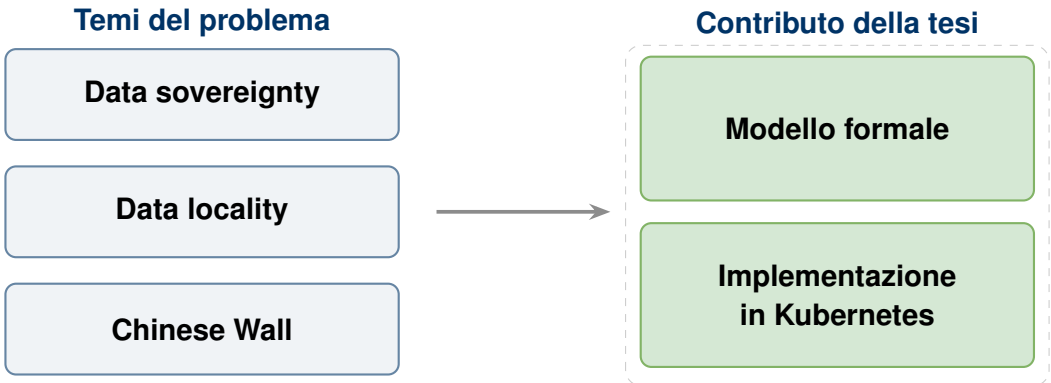
Enti esterni sottomettono i propri task al cluster dell'organizzazione che detiene i dati.

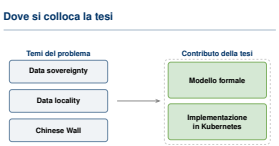


Partiamo dal contesto generale. Spesso, chi possiede dei dati vuole renderli **utilizzabili ad altri senza cederli direttamente**: ad esempio un consorzio di ospedali che mette dati clinici a disposizione di gruppi di ricerca esterni.

L'idea è quindi **portare la computazione dove si trovano i dati**. Come in figura, enti esterni sottomettono i task, e l'organizzazione li esegue sui nodi del proprio cluster che ospitano i dataset, senza che gli enti conoscano la struttura del cluster o decidano dove eseguirli.

Dove si colloca la tesi



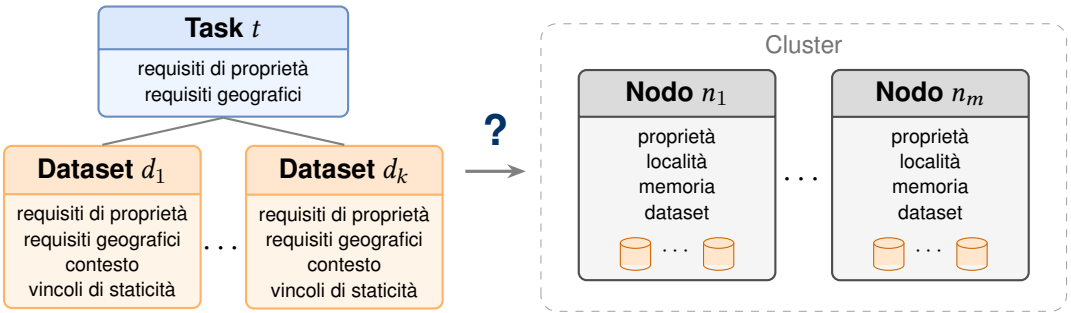


Questo problema tocca tre ambiti di ricerca noti:

- la **data sovereignty**, da cui derivano determinati vincoli geografici;
- la **data locality**, per portare il calcolo vicino ai dati, sia per risparmiare banda sia perché certi dati non si possono trasferire;
- e infine il modello di **Chinese Wall**, per isolare i nodi dalle tracce che i task lasciano in memoria.

La tesi li riunisce in un unico meccanismo di scelta del nodo, con un modello formale che li traduce in politiche e un'implementazione concreta in Kubernetes.

Il problema: dove eseguire il task?



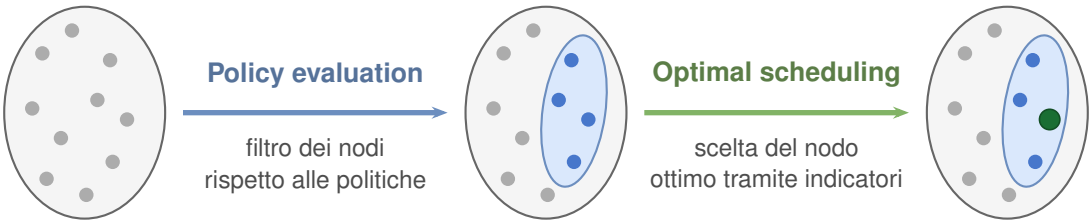


Ora guardiamo il problema più da vicino. Da una parte ci sono **i nodi**, ognuno con un livello per ogni proprietà, una località e i dataset che ospita, che possono comunque essere trasferiti altrove.

Dall'altra c'è un **task che richiede dei dataset**, e bisogna **scegliere** su quale nodo eseguirlo. Task e dataset dichiarano dei requisiti: un livello minimo per ogni proprietà e le aree geografiche in cui possono trovarsi. I dataset hanno inoltre un contesto, cioè l'ambito o l'organizzazione a cui appartengono, e possono essere statici, cioè non trasferibili.

Il problema **non è banale** perché questi vincoli arrivano da entità diverse, e il nodo scelto deve rispettarli tutti contemporaneamente.

Il modello: prima si filtra, poi si sceglie





Per scegliere il nodo su cui eseguire il task, il modello segue due fasi. Nella prima, si parte da tutti i nodi del cluster per poi **filtrarli** rispetto ad un **insieme di politiche** che devono rispettare per poter rimanere nell'insieme dei nodi ammissibili.

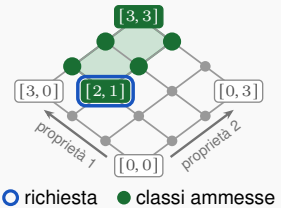
Poi, nella seconda, ad ogni nodo ammissibile viene **assegnato un punteggio** sulla base di un insieme di indicatori, che ne valutano l'ottimalità secondo determinate metriche. I punteggi vengono infine combinati selezionando il nodo con il punteggio più alto.

Ora vediamo le politiche in questione, partendo da quelle statiche.

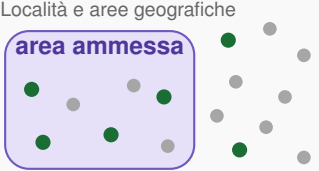
Politiche statiche

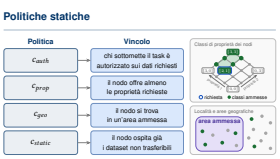
Politica	Vincolo
c_{auth}	chi sottomette il task è autorizzato sui dati richiesti
c_{prop}	il nodo offre almeno le proprietà richieste
c_{geo}	il nodo si trova in un'area ammessa
c_{static}	il nodo ospita già i dataset non trasferibili

Classi di proprietà dei nodi



Località e aree geografiche



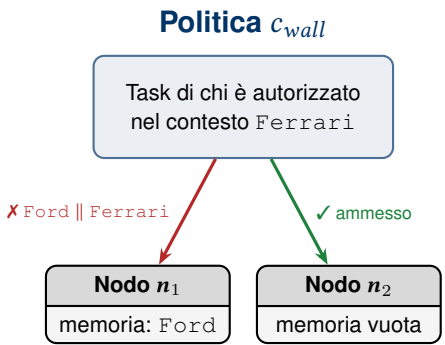


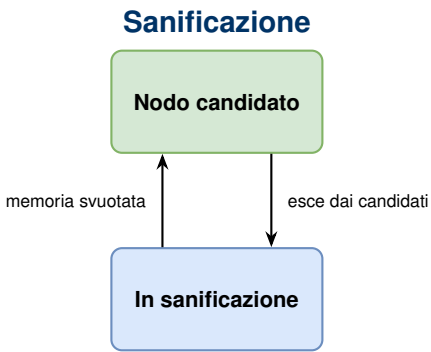
Le politiche statiche, cioè che non dipendono dallo stato del cluster, sono quattro. La prima, c_{auth}, richiede che chi sottomette il task sia **autorizzato** sui dati che richiede, altrimenti il task viene rifiutato subito.

Poi c_{prop} richiede che il nodo candidato abbia **almeno il livello richiesto** su ogni proprietà. I livelli di un nodo formano la sua classe di proprietà, mentre il task, insieme ai suoi dataset, richiede una classe minima; ordinando le classi si ottiene una vista a reticolo come in figura, e il nodo è ammesso per quel task solo se la sua classe domina quella richiesta.

Infine, c_{geo} richiede che il nodo candidato si trovi in un'**area geografica ammessa** sia dal task sia dai suoi dataset, e c_{static} che ospiti già i dataset **non trasferibili** richiesti.

La politica dinamica: Chinese Wall sui nodi







Arriviamo alla politica dinamica, che riprende il modello di **Chinese Wall** per i conflitti di interesse, applicato ai nodi.

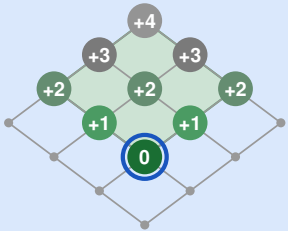
Ogni esecuzione può lasciare sul nodo **tracce dei dati elaborati** che un task successivo potrebbe leggere. Per questo ogni nodo mantiene una **memoria dei contesti** dei dati che ha trattato, e c_{wall} lo esclude se chi sottomette il task è autorizzato su un contesto in conflitto con uno di quelli in memoria. Nell'esempio il primo nodo ha elaborato dati Ford, in conflitto con il contesto Ferrari del task, e viene escluso, mentre il secondo resta disponibile.

Col tempo, però, la memoria si riempirebbe e i nodi finirebbero esclusi quasi per tutti. Per evitare questa **starvation**, ogni nodo viene periodicamente sanificato e la sua memoria svuotata.

Gli indicatori: scegliere tra i nodi ammissibili

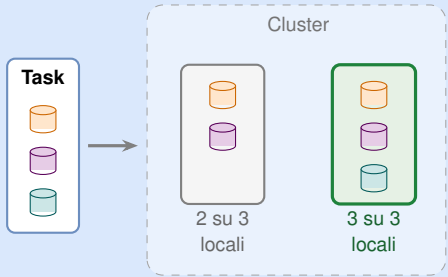
Minimo privilegio

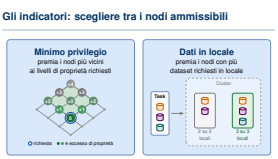
premia i nodi più vicini
ai livelli di proprietà richiesti



Dati in locale

premia i nodi con più
dataset richiesti in locale



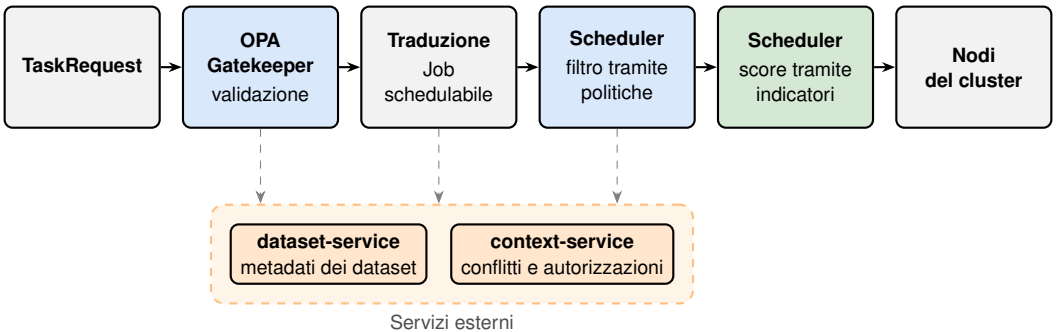


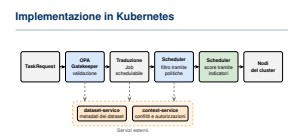
Tra i nodi ammissibili, si sceglie quello ottimo con due indicatori. Il primo segue il principio del **minimo privilegio**, premiando i nodi che superano di meno i livelli richiesti, così da non occupare risorse più preziose del necessario.

Il secondo, invece, premia i nodi che hanno **più dataset richiesti in locale** per ridurre i trasferimenti e la banda consumata. La somma pesata dei due punteggi andrà a decidere il nodo da assegnare al task.

A questo punto il modello è completo.

Implementazione in Kubernetes



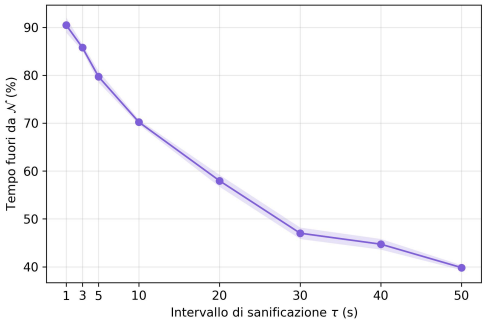


Per l'implementazione ho sfruttato i **meccanismi di estensione** che offre Kubernetes, sviluppando una pipeline che segue le due fasi del modello.

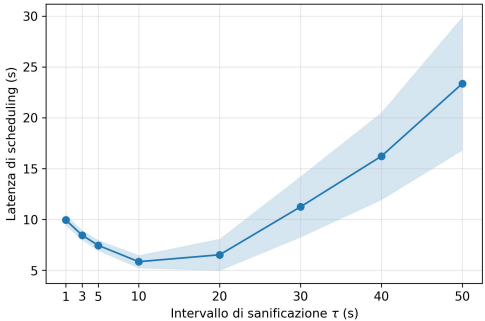
Il task arriva come **risorsa Kubernetes**, viene validato da un controller di ammissione, OPA Gatekeeper, e poi tradotto in un Job **schedulabile**. Lo scheduler nativo esteso **filtra** quindi i nodi ammissibili tramite le politiche, scegliendo infine quello con il **punteggio migliore** rispetto agli indicatori.

Inoltre, lungo il percorso vengono interrogati due servizi esterni, uno per i metadati dei dataset e uno per conflitti e autorizzazioni.

Valutazione: disponibilità dei nodi e latenza dei task

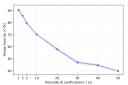


Tempo in cui un nodo è fuori dai candidati

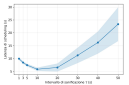


Latenza media di scheduling di un task

Valutazione: disponibilità dei nodi e latenza dei task



Tempo in cui un nodo è fuori dai candidati



Latenza media di scheduling di un task

Nella valutazione mi sono concentrato soprattutto sulla **componente dinamica** del modello, facendo variare ogni quanto i nodi vengono sanificati.

Ne emerge un **compromesso** tra la disponibilità dei nodi e la latenza di scheduling dei task. Infatti, sanificare **spesso** toglie i nodi dai candidati più a lungo, come si vede nel grafico a sinistra, mentre sanificare di **rado** può lasciare i task bloccati a lungo a causa delle tracce in conflitto sui nodi, come mostra la latenza a destra.

Conclusioni e sviluppi futuri

Conclusioni

- Modello formale di scheduling guidato da politiche
- Prototipo che lo realizza in Kubernetes

Sviluppi futuri

- Task con semantica propria e dati reali
- Strategie ottimizzate di sanificazione
- Estensione multi-cluster



Per concludere, la tesi propone un **modello formale** di scheduling guidato da politiche, che rispetta i vincoli di protezione dei dati, e un **prototipo** in Kubernetes che lo realizza.

Come sviluppi futuri, il primo è usare task e dati **reali**, dato che per ora il prototipo li simula a livello logico. Il secondo è adottare strategie di sanificazione **ottimizzate** per migliorare il compromesso analizzato. E per finire, estendere il sistema in ambiente **multi-cluster**.

Grazie per l'attenzione

Luca Milanesi

Università degli Studi di Milano

Grazie per l'attenzione

Luca Milanesi
Università degli Studi di Milano

Vi ringrazio per l'attenzione, sono a disposizione per eventuali domande.